

"""

Пересыльщик банковских уведомлений (userbot).

ЗАЧЕМ ОН НУЖЕН. Уведомления о поступлении денег приходят от БОТА банка. Telegram не доставляет сообщения одного бота другому — значит, наш магазин-бот такие уведомления увидеть не может в принципе, сколько его в тот чат ни добавляй. Этот сервис заходит в Telegram ТВОИМ аккаунтом (обычным пользователем, не ботом), читает чат с ботом банка и пересылает текст магазин-боту как обычное сообщение — такое бот уже видит и разбирает.

ЧТО ОН ДЕЛАЕТ И ЧЕГО НЕ ДЕЛАЕТ:

- слушает ОДИН заданный чат (SOURCE_CHAT) и пересылает только его сообщения;
- ничего не отправляет от твоего имени никуда больше, ничего не читает в других чатах, не хранит переписку;
- вход в аккаунт (сессия) хранится файлом на диске самого сервиса, никуда не копируется и не показывается; страница настройки закрыта паролем (RELAY_ADMIN_TOKEN).

ВХОД В АККАУНТ выполняется один раз через веб-страницу этого же сервиса — компьютер не нужен: открываешь адрес сервиса на телефоне, вводишь номер и код из Telegram, потом выбираешь в списке чат с ботом банка. Всё.

"""

```
import asyncio
import os
```

```
from aiohttp import web
from telethon import TelegramClient, events
from telethon.sessions import StringSession
```

```
API_ID = int(os.getenv("TG_API_ID", "0") or "0")
API_HASH = os.getenv("TG_API_HASH", "")
SESSION = os.getenv("TG_SESSION", "")
```

```
# Куда сохраняем вход и выбранный чат. Это ДИСК сервиса (том Railway), а не
# переменные окружения: так строку сессии — а это доступ к аккаунту — не нужно
# никуда копировать руками и некуда случайно переслать.
DATA_DIR = os.getenv("SESSION_DIR", "/data")
SESSION_FILE = os.path.join(DATA_DIR, "relay")          # Telethon добавит .session
SOURCE_FILE = os.path.join(DATA_DIR, "source_chat.txt")
TARGET_FILE = os.path.join(DATA_DIR, "target_chat.txt")

# Чат, ОТКУДА читаем (бот банка): @username или числовой id
```

```

SOURCE_CHAT = os.getenv("SOURCE_CHAT", "@CardXabarBot")
# Кому пересылаем: @username магазин-бота или id группы, где он админ
TARGET_CHAT = os.getenv("TARGET_CHAT", "@osonstoreuz_bot")

# Пароль к странице настройки (без него страницу открывать нельзя)
ADMIN_TOKEN = os.getenv("RELAY_ADMIN_TOKEN", "")

PORT = int(os.getenv("PORT", "8080"))

client: TelegramClient | None = None
_state = {"authorized": False, "listening": False, "last_forward": None, "me": None, "login": None}
_login = {"phone": None, "hash": None}

def _load_choice(path: str, fallback: str) -> str:
    """Выбранный на странице чат. Файл важнее переменной окружения: его
    человек задаёт сам, одной кнопкой, без правки настроек Railway."""
    try:
        with open(path, encoding="utf-8") as f:
            saved = f.read().strip()
            if saved:
                return saved
    except OSError:
        pass
    return fallback

def _save_choice(path: str, value: str) -> None:
    os.makedirs(DATA_DIR, exist_ok=True)
    with open(path, "w", encoding="utf-8") as f:
        f.write(value.strip())

def _load_source() -> str:
    return _load_choice(SOURCE_FILE, SOURCE_CHAT)

def _load_target() -> str:
    return _load_choice(TARGET_FILE, TARGET_CHAT)

def _new_client() -> TelegramClient:
    # С TG_SESSION — строкой из переменной (запасной путь), иначе — файл на
    # диске сервиса, который переживает передеплой.
    if SESSION:
        return TelegramClient(StringSession(SESSION), API_ID, API_HASH)

```

```

os.makedirs(DATA_DIR, exist_ok=True)
return TelegramClient(SESSION_FILE, API_ID, API_HASH)

def _match_source(chat_id: int, username: str | None) -> bool:
    src = (_load_source() or "").strip()
    if not src:
        return False
    if src.startswith("@"):
        return (username or "").lower() == src[1:].lower()
    try:
        return int(src) == chat_id
    except ValueError:
        return False

async def _start_listening():
    """Вешает обработчик на новые сообщения и запоминает, что слушаем."""
    global _state
    if not client or not await client.is_user_authorized():
        return
    if _state["listening"]:
        # Уже слушаем: обработчик вешается ОДИН раз. Иначе после входа через
        # страницу и повторного вызова из главного цикла каждое уведомление
        # пересылалось бы дважды.
        return

    me = await client.get_me()
    _state["me"] = {"id": me.id, "username": me.username, "name": me.first_name}
    _state["authorized"] = True

    @client.on(events.NewMessage())
    async def _on_message(event):
        try:
            chat = await event.get_chat()
            username = getattr(chat, "username", None)
            if not _match_source(event.chat_id, username):
                return # чужой чат — не наше дело, молча выходим

            text = event.message.message or ""
            if not text.strip():
                return

            await client.send_message(_load_target(), text)
            _state["last_forward"] = text[:200]
            print(f"[relay] переслано ({len(text)} симв.): {text[:80]!r}", flush=

```

```

except Exception as e:
    _state["error"] = str(e)
    print(f"[relay] ошибка пересылки: {e}", flush=True)

_state["listening"] = True
print(f"[relay] слушаю {_load_source()} -> пересылаю в {_load_target()}", flu

# ----- Веб-страница настройки -----

PAGE_CSS = """
<style>
body{background:#0b0c0f;color:#fff;font-family:-apple-system,system-ui,sans-seri
.box{max-width:460px;margin:0 auto}
.card{background:rgba(255,255,255,.05);border:1px solid rgba(255,255,255,.1);bor
h1{font-size:20px;margin:0 0 4px} h2{font-size:15px;margin:0 0 10px}
.muted{color:#9aa0a8;font-size:13px}
input{width:100%;box-sizing:border-box;background:rgba(255,255,255,.06);border:1
color:#fff;border-radius:12px;padding:12px;font-size:16px;margin:6px 0 12p
button{width:100%;background:linear-gradient(135deg,#2AABEE,#7B7FE0);color:#fff;
padding:14px;font-size:15px;font-weight:600}
code{background:rgba(255,255,255,.08);padding:2px 6px;border-radius:6px;font-siz
.ok{color:#34d399} .bad{color:#f87171}
table{width:100%;font-size:13px;border-collapse:collapse} td{padding:6px 0;borde
</style>
"""

def _page(body: str) -> web.Response:
    return web.Response(
        text=f"<!doctype html><html><head><meta charset=utf-8>"
            f"<meta name=viewport content='width=device-width,initial-scale=1'>"
            f"<title>SMS Relay</title>{PAGE_CSS}</head><body><div class=box>{bod
        content_type="text/html",
    )

def _check_token(request) -> bool:
    if not ADMIN_TOKEN:
        return False
    return request.query.get("token") == ADMIN_TOKEN or request.cookies.get("rela

async def handle_index(request: web.Request) -> web.Response:
    if not ADMIN_TOKEN:
        return _page("<div class=card><h1>Не настроено</h1><p class=muted>Задай п

```

```

        "<code>RELAY_ADMIN_TOKEN</code> – это пароль к этой странице
if not _check_token(request):
    return _page("<div class=card><h1>Доступ закрыт</h1><p class=muted>Открой
        "<code>?token=ТВОЙ_RELAY_ADMIN_TOKEN</code></p></div>")

if not API_ID or not API_HASH:
    return _page("<div class=card><h1>Нет ключей Telegram</h1><p class=muted>
        "<code>my.telegram.org</code> → API development tools и впиш
        "<code>TG_API_ID</code> и <code>TG_API_HASH</code>.</p></div>")

authorized = client is not None and await client.is_user_authorized()

if authorized:
    me = _state.get("me") or {}
    source = _load_source()
    target = _load_target()
    session_note = (
        "Вход хранится в переменной <code>TG_SESSION</code>."
        if SESSION else
        "Вход сохранён на диске сервиса – копировать ничего не нужно, он пере
    )
    source_block = (
        f"<tr><td>Источник</td><td><code>{source}</code></td></tr>"
        if source else
        "<tr><td>Источник</td><td class=bad>не выбран</td></tr>"
    )
    return _page(
        "<div class=card><h1 class=ok>Аккаунт подключён</h1>"
        f"<p class=muted>{me.get('name','')} @ {me.get('username','')} · id {m
        + (" if source else
            "<div class=card><h2>Шаг 1 из 2</h2><p class=muted>Выбери чат, ИЗ
            f"пересылать уведомления банка.</p><a href='/dialogs?token={ADMIN_
            "<button>Откуда пересылать</button></a></div>")
        + (" if target else
            "<div class=card><h2>Шаг 2 из 2</h2><p class=muted>Выбери своего м
            f"КУДА пересылать.</p><a href='/dialogs?token={ADMIN_TOKEN}&for=ta
            "<button>Куда пересылать</button></a></div>")
        + "<div class=card><h2>Что слушаем</h2><table>"
        + source_block
        + (f"<tr><td>Куда шлём</td><td><code>{target}</code></td></tr>" if ta
            else "<tr><td>Куда шлём</td><td class=bad>не выбран</td></tr>")
        + f"<tr><td>Статус</td><td>{'🟢 слушаю' if _state['listening'] else '
        f"<tr><td>Последняя пересылка</td><td>{_state.get('last_forward') or
        f"<tr><td>Ошибка</td><td class=bad>{_state.get('error') or '-'}</td><
        "</table></div>"
        f"<div class=card><h2>Вход</h2><p class=muted>{session_note}</p></div>")

```

```

        f"<div class=card><a href='/dialogs?token={ADMIN_TOKEN}&for=source'><
        f"<div style='height:10px'></div>"
        f"<a href='/dialogs?token={ADMIN_TOKEN}&for=target'><button>Сменить п
        f"<div class=card><form method=post action='/test?token={ADMIN_TOKEN}
        "<h2>Проверка</h2><p class=muted>Отправит тестовое уведомление магази
        "<button>Отправить тест</button></form></div>"
    )

# Не авторизованы — шаг 1 (телефон) или шаг 2 (код)
if _login["hash"]:
    return _page(
        "<div class=card><h1>Код из Telegram</h1>"
        f"<p class=muted>Код отправлен на {_login['phone']} — он придёт СООБЩ
        "не по SMS. Если включён облачный пароль (2FA), впиши и его.</p>"
        f"<form method=post action='/sign_in?token={ADMIN_TOKEN}'>"
        "<input name=code placeholder='Код (12345)' inputmode=numeric autocom
        "<input name=password type=password placeholder='Облачный пароль (есл
        "<button>Войти</button></form></div>"
    )

return _page(
    "<div class=card><h1>Вход в аккаунт</h1>"
    "<p class=muted>Номер того аккаунта, КУДА приходят уведомления банка. "
    "Формат: +998901234567</p>"
    f"<form method=post action='/send_code?token={ADMIN_TOKEN}'>"
    "<input name=phone placeholder='+998901234567' inputmode=tel>"
    "<button>Получить код</button></form></div>"
)

async def handle_send_code(request: web.Request) -> web.Response:
    if not _check_token(request):
        return _page("<div class=card><h1>Доступ закрыт</h1></div>")
    data = await request.post()
    phone = (data.get("phone") or "").strip()
    try:
        sent = await client.send_code_request(phone)
        _login["phone"], _login["hash"] = phone, sent.phone_code_hash
    except Exception as e:
        return _page(f"<div class=card><h1 class=bad>Не удалось отправить код</h1>
            f"<p><a style='color:#2AABEE' href='/?token={ADMIN_TOKEN}'>Н
        raise web.HTTPFound(f"/?token={ADMIN_TOKEN}")

async def handle_sign_in(request: web.Request) -> web.Response:
    if not _check_token(request):

```

```

        return _page("<div class=card><h1>Доступ закрыт</h1></div>")
data = await request.post()
code = (data.get("code") or "").strip()
password = (data.get("password") or "").strip()

try:
    await client.sign_in(phone=_login["phone"], code=code, phone_code_hash=_1
except Exception as e:
    # Включён облачный пароль (2FA) – Telethon просит его отдельным шагом
    if "password" in str(e).lower() and password:
        try:
            await client.sign_in(password=password)
        except Exception as e2:
            return _page(f"<div class=card><h1 class=bad>Неверный пароль</h1>
                f"<p><a style='color:#2AABEE' href='/?token={ADMIN_T
    else:
        return _page(f"<div class=card><h1 class=bad>Не вошли</h1><p class=mu
            f"<p><a style='color:#2AABEE' href='/?token={ADMIN_TOKEN

_login["hash"] = None
await _start_listening()
print("[relay] СТОПА СЕССИИ (впиши в TG_SESSION):", client.session.save(), f
raise web.HTTPFound(f"/?token={ADMIN_TOKEN}")

async def handle_dialogs(request: web.Request) -> web.Response:
    """Список последних чатов – тут же кнопкой выбирается тот, что слушаем.
    Так человеку не нужно лезть в настройки Railway и куда-то вписывать id."""
    if not _check_token(request):
        return _page("<div class=card><h1>Доступ закрыт</h1></div>")
    if not await client.is_user_authorized():
        raise web.HTTPFound(f"/?token={ADMIN_TOKEN}")

    mode = request.query.get("for", "source")
    current = _load_source() if mode == "source" else _load_target()
    rows = ""
    async for dialog in client.iter_dialogs(limit=50):
        name = (dialog.name or "-").replace("<", "").replace(">", "")
        uname = getattr(dialog.entity, "username", None)
        value = ("@" + uname) if uname else str(dialog.id)
        mark = " 

```

```

        f"<button style='padding:8px;font-size:13px'>Выбрать</button></form><
    )
    title = "Откуда пересылать" if mode == "source" else "Куда пересылать"
    hint = ("Найди бота банка — тот, что присылает уведомления о поступлении."
           if mode == "source" else
           "Найди СВОЕГО магазин-бота — того, где мини-апп с товарами.")
    return _page(f"<div class=card><h2>{title}</h2>"
                f"<p class=muted>{hint} Нажми «Выбрать» напротив него.</p>"
                f"<table>{rows}</table>"
                f"<p><a style='color:#2AABEE' href='/?token={ADMIN_TOKEN}'>Назад

async def handle_pick(request: web.Request) -> web.Response:
    """Сохраняет выбранный чат-источник на диск сервиса."""
    if not _check_token(request):
        return _page("<div class=card><h1>Доступ закрыт</h1></div>")
    data = await request.post()
    value = (data.get("value") or "").strip()
    field = (data.get("field") or "source").strip()
    if value:
        _save_choice(SOURCE_FILE if field == "source" else TARGET_FILE, value)
        print(f"[relay] {field} выбран: {value}", flush=True)
    raise web.HTTPFound(f"/?token={ADMIN_TOKEN}")

async def handle_test(request: web.Request) -> web.Response:
    if not _check_token(request):
        return _page("<div class=card><h1>Доступ закрыт</h1></div>")
    try:
        await client.send_message(
            _load_target(),
            "🟢 Perevod na kartu\n➕ 1.00 UZS\n🏠 ***0000\n📍 TEST RELAY\n💰 0.0
        )
        msg = "<h1 class=ok>Тест отправлен</h1><p class=muted>Проверь, дошло ли с
    except Exception as e:
        msg = f"<h1 class=bad>Не отправилось</h1><p class=muted>{e}</p>"
    return _page(f"<div class=card>{msg}<p><a style='color:#2AABEE' href='/?token

async def handle_health(request: web.Request) -> web.Response:
    me = _state.get("me") or {}
    return web.json_response({
        "authorized": _state["authorized"], "listening": _state["listening"],
        "source": _load_source(), "target": _load_target(),
        "me_id": me.get("id"), "me_username": me.get("username"),
        "last_forward_at": bool(_state.get("last_forward")), "error": _state.get(

```

```
}}
```

```
async def main():
    global client
    client = _new_client()
    await client.connect()

    if await client.is_user_authorized():
        await _start_listening()
    else:
        print("[relay] аккаунт не подключён — открой веб-страницу сервиса и войди")

    app = web.Application()
    app.router.add_get("/", handle_index)
    app.router.add_post("/send_code", handle_send_code)
    app.router.add_post("/sign_in", handle_sign_in)
    app.router.add_get("/dialogs", handle_dialogs)
    app.router.add_post("/pick", handle_pick)
    app.router.add_post("/test", handle_test)
    app.router.add_get("/health", handle_health)

    runner = web.AppRunner(app)
    await runner.setup()
    await web.TCPSite(runner, "0.0.0.0", PORT).start()
    print(f"[relay] веб-страница настройки поднята на порту {PORT}", flush=True)

    # БАГ БЫЛ ЗДЕСЬ: run_until_disconnected() вызывался сразу, даже когда в
    # аккаунт ещё не вошли — Telethon спрашивал у Telegram состояние обновлений,
    # получал AuthKeyUnregistered и весь процесс падал. Вместе с ним падала и
    # страница входа, то есть войти было невозможно в принципе.
    #
    # Теперь ждём входа: пока его нет — просто держим страницу живой, как
    # только вход появился (человек ввёл код на странице) — начинаем слушать.
    while True:
        try:
            if await client.is_user_authorized():
                await _start_listening()
                await client.run_until_disconnected()
                # Сюда попадаем, если соединение разорвалось — переподключаемся
                print("[relay] соединение с Telegram разорвано, переподключаюсь",
                      _state["listening"] = False)
                await client.connect()
            else:
                await asyncio.sleep(5)
        except Exception as e:
```

```
        _state["error"] = str(e)
    print(f"[relay] ошибка главного цикла: {e}", flush=True)
    _state["listening"] = False
    await asyncio.sleep(5)
    try:
        await client.connect()
    except Exception:
        pass

if __name__ == "__main__":
    asyncio.run(main())
```